

Pulse Assessment - Explainer

How scoring works, where the numbers come from, and how to use the app in the field. Version 2.5.3 - June 2026.

1. Where do the benchmarks come from?

There are no external benchmarks. Pulse does not compare against industry data, peer companies, or a curated Dynatrace dataset. Every number you see is computed **live, from the customer's own tenant, using DQL queries against Grail.**

Each of the ~94 criteria is a **ratio between two DQL results** the app runs on the fly:

```
criterion (%) = (numerator DQL) / (denominator DQL) x 100
```

Concrete example - i1, Host CPU coverage:

- Numerator: `timeseries val=avg(dt.host.cpu.usage), by:{dt.entity.host} | dedup | count -- hosts reporting CPU metrics`
- Denominator: `fetch dt.entity.host | summarize count() -- total known hosts`
- Result: `87 / 100 = 87%`

That percentage is then compared against **thresholds hardcoded in the app** (for example `[{min: 90}, {min: 50}, {min: 1}]`). Those thresholds were defined by the developer (the SE author) based on field experience - they do **not** come from a statistical baseline. They live in `ui/app/queries.ts` and `ui/app/data/criterionTiers.ts`.

In one sentence: the "benchmark" is the customer's own tenant, scored against reasonable targets defined by the SE.

2. What is the use case? Customer-facing? QBRs?

Pulse is designed for three SE-led moments. It is an **SE tool**, not an official Dynatrace product - no SLA, no support.

Moment	How to use it
Before a QBR / EBR	Run Pulse against the customer tenant, export scores, bring slides showing "before vs. after" coverage and actionable gaps.
Health-check / Adoption review	Show the customer where the platform is under-utilized (e.g. 30% of hosts without logs, AI Observability at 0%). Generate enablement follow-ups.
Land-and-expand	Identify capabilities with low coverage to justify new licenses (Log Management, RUM, AppSec).

Can I show it directly to the customer?

Yes - with care. The default UI is presentable in front of a customer (radar, capability cards, business-language remediations, optional DPS-cost badge). Internal diagnostics (perf JSON download, force-refresh) are gated behind ?dev=1 in the URL and stay hidden by default. So in a QBR you can share-screen safely and the customer only sees the executive view.

3. Coverage vs. Maturity - what is the difference?

Both scores are computed from the same ~94 criteria but aggregate them differently.

Coverage Score (simple average)

"Of all the criteria, how many passed?"

```
coverage = (criteria that passed their threshold) / (total criteria) x 100
```

- Each criterion is worth **1 point** if it crosses the minimum threshold, **0** otherwise.
- No weighting by importance or ordering.
- This is the number shown in the **radar chart**.

Maturity Score (tier-weighted, progressive)

"How deep is the adoption?"

Every criterion is classified into one of three tiers (see `criterionTiers.ts`):

Tier	Meaning	Weight
Foundation	Essential - everyone should have this	60%
Best Practice	Deeper adoption	25%
Excellence	Advanced / mature usage	15%

A **progressive rule** then prevents the customer from skipping to advanced criteria before the basics are solid (use `CoverageData.ts` lines 747-750):

```
maturityScore =  
  foundation% x 60  
+ bestPractice% x 25  ← only counts if foundation >= 80%  
+ excellence%   x 15  ← only counts if bestPractice >= 60%
```

Maturity bands

>=80 Excellent • >=60 Good • >=40 Moderate • >=20 Low • <20 N/A

Maturity levels (L0-L3)

Level	Label	Rule
L0	Not Adopted	Foundation < 50%
L1	Foundation	Foundation >= 50%
L2	Operational	Foundation 100% + Best Practice >= 50%
L3	Optimized	Foundation 100% + Best Practice 100% + Excellence >= 50%

Why both numbers matter

A customer with 70% **coverage** can still have low **maturity** if they passed many Excellence-tier criteria while missing Foundation ones. Coverage tells you *"how much is turned on"*; Maturity tells you *"whether the base is solid before the advanced stuff"*.

Coverage = breadth. Maturity = depth + ordering.

References in source: `ui/app/queries.ts` (criteria + thresholds), `ui/app/data/criterionTiers.ts` (tier classification), `ui/app/hooks/useCoverageData.ts` (score computation, lines 731-766). See `docs/MEMORY.md` for the full logic and DQL reference.